

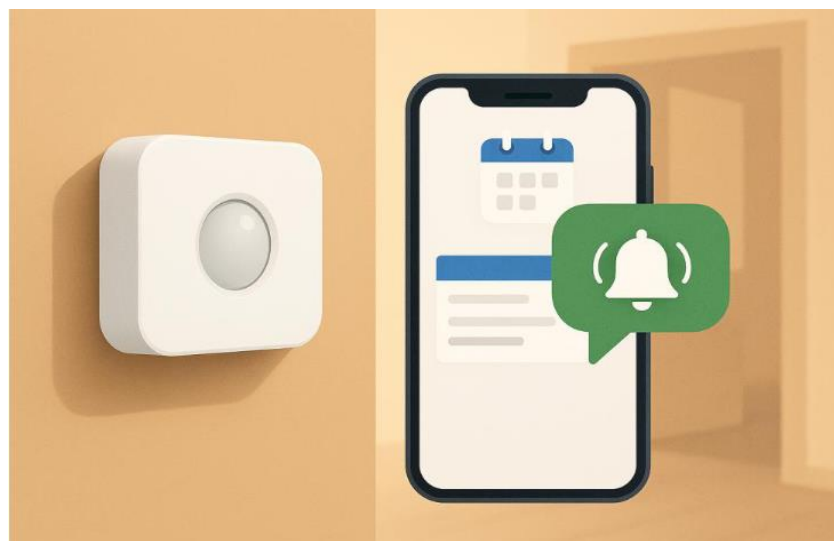
成果発表

GROUP4

何を作ったか

システムの概要

- Nature Remo 3 の人感センサー、Google カレンダー、LINE を連携させ、予定時刻に応じた通知を送信する
- 出発すべき時刻を過ぎても外出が確認できない場合、LINE メッセージで催促通知を行う



製品の機能

- Nature Remo 3 の人感センサーを利用し、Nature API を通じて取得した情報からユーザの外出状況を自動で判定する
- Google Calendar API と連携して当日の予定情報を取得し、場所情報やキーワードから外出予定かどうかを判定する
- 予定開始時刻が近づいている、または開始後も外出が確認できない場合、LINE Messaging API を利用して段階的に通知メッセージを送信する
- ユーザは LINE 上でスヌーズや通知キャンセルを行うことができ、システムはその内容を反映して通知状態を管理する
- システムは定期的にヘルスチェック通知を送信し、正常に稼働していることを確認できる

想定する利用者

外出予定を忘れがちな人や、時間にルーズで出発が遅れがちな一人暮らしの人が対象

特に

- 自宅でだらだらして出発が遅れがちな人
- 子供や学生で、塾・習い事などの予定に遅刻しがちな人
- 一人暮らしで出発を催促してくれる人がいない人



スケジュールと設計

作成方法と設計

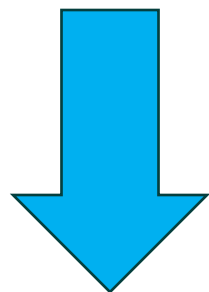
- 外部機能とのやり取りの単位でモジュールを切り出し、複数人に割り当てることで並列作業を実現した
- 各モジュールで動作確認を行ったのち、結合、リファクタリングを行った
- 設計書の作成にはAIを用い、mermaid記法で記述することで、チーム内でのロジックのすり合わせを行なった。

モジュール分割後のスケジュール

タスク	担当	4/24 5限	5/1	5/8 5限	5/14 5限	5/21 5限
要求仕様・設計のみなおし	全員	■	■	■	■	■
データ保存・状態管理ロジック	加藤		■			
NatureModuleの実装	加藤		■			
CalendarModuleの実装	藤井		■			
StateModuleの実装	丸本			■		
NotificationModuleの実装	滝			■		
システムテスト	全員	■	■	■	■	■
成果発表資料作成	全員	■	■	■	■	■

リファクタリング

完成したモジュールは個々で状態管理や設定に関する記述が含まれていたため、テストやコードの修正が容易でなかった。



結合・リファクタリング

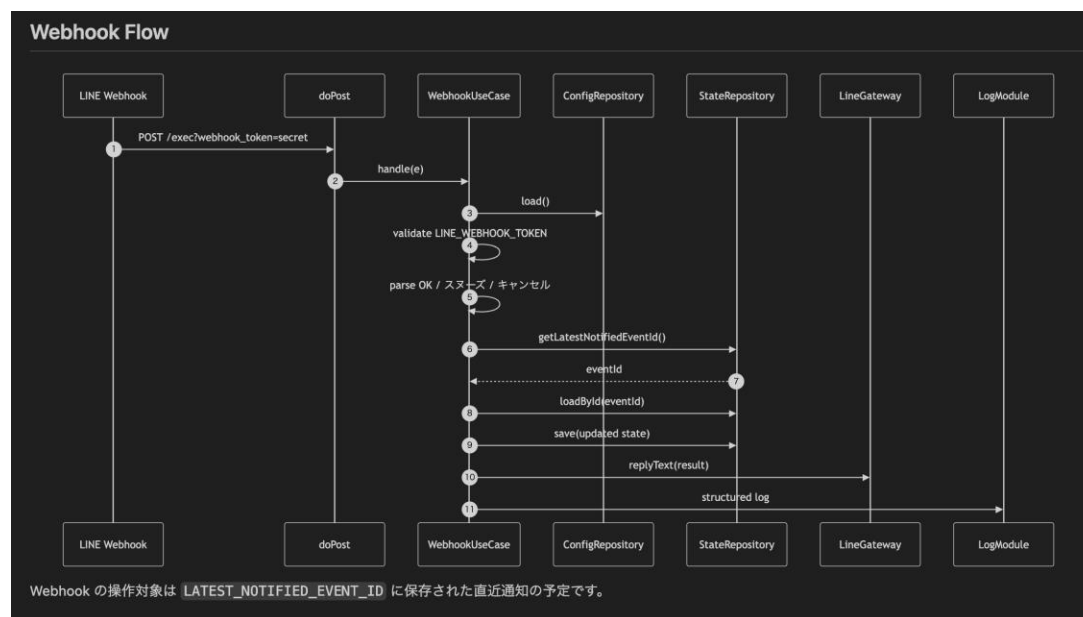
設定ファイルやロジック、ドメインモデル、をモジュールから抜き出して集約、共通化し、テストのしやすさ、可読性を向上した。

設計書の例

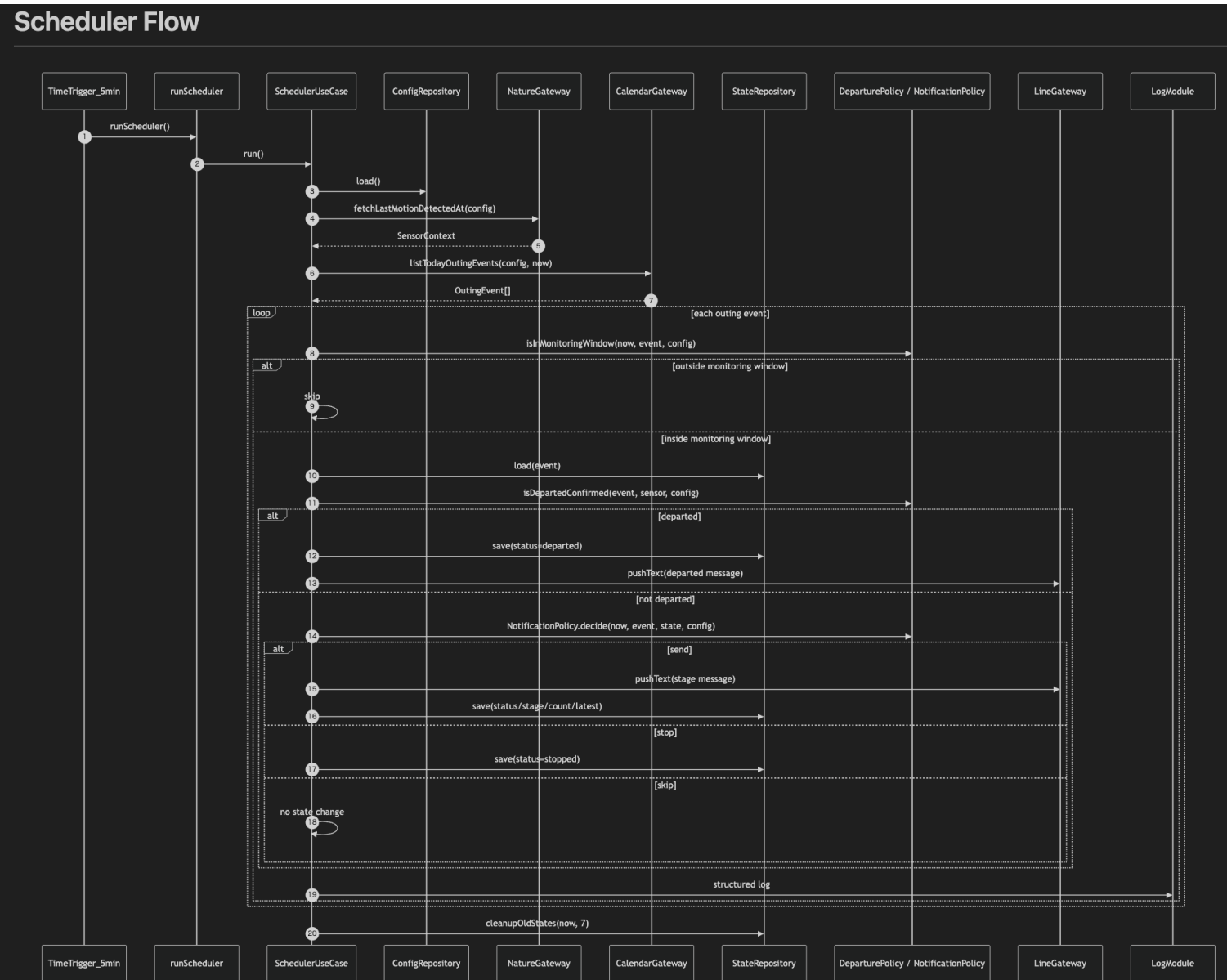
- シーケンス図にすることで視覚的に理解しやすくなった。
- 設計では、UseCase層, Domain層, Gateway・Repository層というようなレイヤードアーキテクチャを採用し、要求を満たすようにコードの改変を楽に行えた

DIとかはやってないからクリーンアーキテクチャとかではないけど

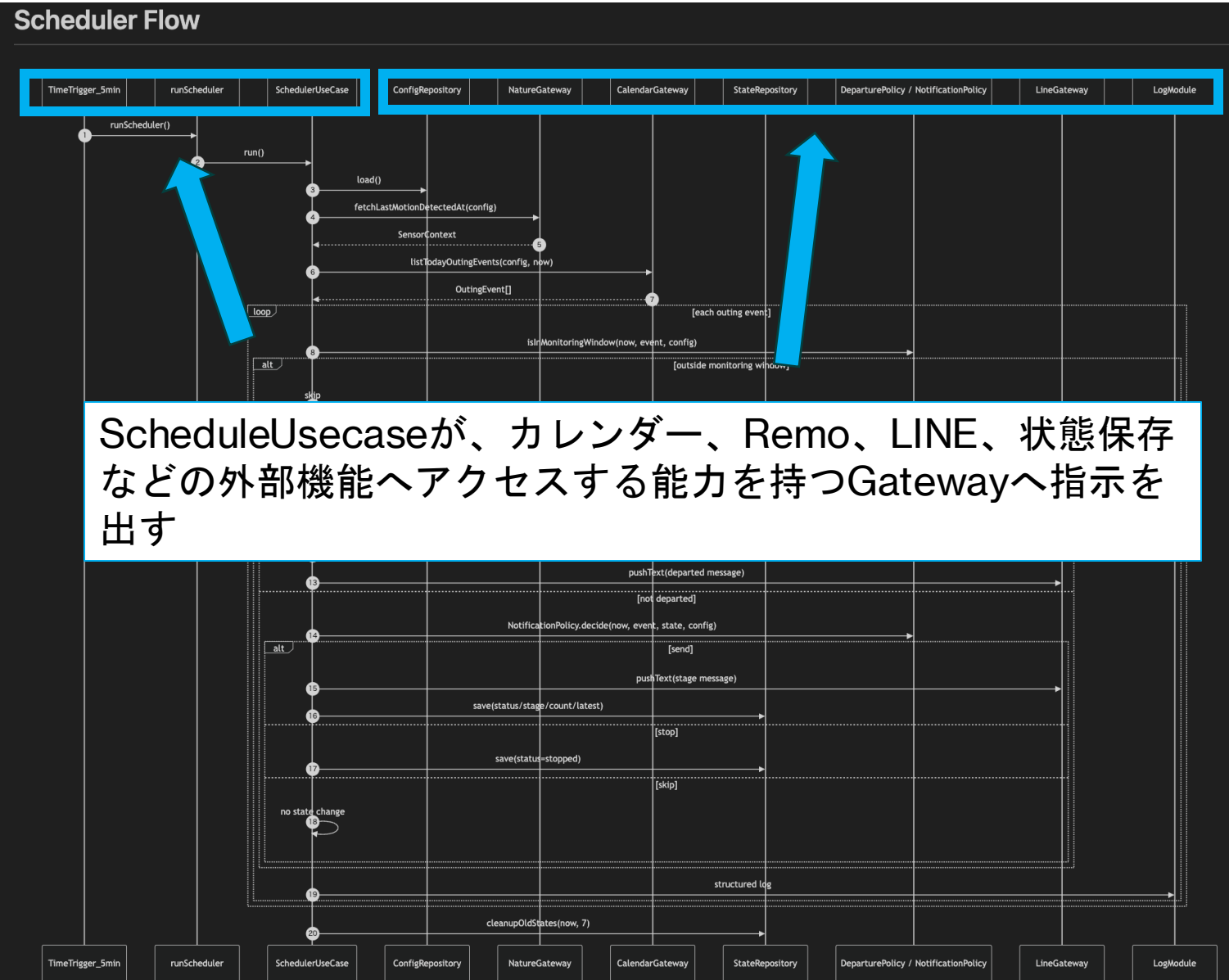
LINEAPIとのやり取りとその周辺のビジネスロジックをまとめたシーケンス図



実装例(5分毎の外出判定)



実装例(5分毎の外出判定)



実装例(5分毎の外出判定)

```
/**
 * 5分ごとの外出判定UseCase。
 */
const SchedulerUseCase = {
  run: function() {
    const traceId = LogModule.createTraceId();
    const now = new Date();

    try {
      const config = ConfigRepository.load();
      const sensor = SchedulerUseCase.fetchSensor_(config, traceId);
      if (!sensor) {
        return;
      }
    }
  }
};
```

NatureModuleから外出記録
を受け取り、続く処理を行う

```
fetchSensor_: function(config, traceId) {
  try {
    const sensor = NatureGateway.fetchLastMotionDetectedAt(config);
    LogModule.info('sensor', {
      deviceId: sensor.deviceId,
      deviceName: sensor.deviceName,
      lastDetectedAtIso: sensor.lastDetectedAtIso,
      fetchedAtIso: sensor.fetchedAtIso
    }, traceId);
    return sensor;
  } catch (e) {
    LogModule.error('error', {
      category: 'sensor',
      message: e.message,
      stack: e.stack
    }, traceId);
    return null;
  }
},
```

```
/**
 * Nature Remo Cloud API Gateway。
 */
const NatureGateway = {
  listDevices: function(config) {
    const response = UrlFetchApp.fetch('https://api.nature.global/1/devices', {
      method: 'get',
      headers: {
        Authorization: 'Bearer ' + config.natureApiToken
      }
    });
    if (response instanceof Error) {
      throw new Error('Nature Remo API エラー(' + response.message + ')');
    }
    const statusCode = response.getResponseCode();
    if (statusCode !== 200) {
      throw new Error('Nature Remo API エラー(' + statusCode + ')');
    }
    return JSON.parse(response.getContentText() || '[]');
  },

  fetchLastMotionDetectedAt: function(config) {
    const devices = NatureGateway.listDevices(config);
    const targetDevices = NatureGateway.filterTargetDevices_(devices, config);
    let latest = null;

    targetDevices.forEach(function(device) {
      const motionEvent = device.newest_events[0];
      if (!motionEvent || !motionEvent.created_at) {
        return;
      }

      const detectedAt = new Date(motionEvent.created_at);
      if (!latest || detectedAt.getTime() > latest.lastDetectedAt.getTime()) {
        latest = {
          deviceId: device.id,
          deviceName: device.name,
          lastDetectedAt: detectedAt,
          lastDetectedAtIso: detectedAt.toISOString(),
          value: motionEvent.val
        };
      }
    });
  }
};
```

Usecase(Gatewayの関数を組み合わせて処理を行う)

Gateway(外部とのやり取りを含む単純な関数)

デモストレーション

カレンダーに場所が入力され、監視（外出30分前）が始まってから予定開始までに1回はセンサーが反応していたが、その後検知できなくなった場合、**外出済み**とみなしてLINEで通知。

26 カレンダー 今日 < > 2026年 5月

+ 作成

2026年 5月 < >

日 月 火 水 木 金 土

26 27 28 29 30 1 2

3 4 5 6 7 8 9

10 11 12 13 14 15 16

17 18 19 20 21 22 23

24 25 26 27 28 29 30

31 1

予約ページ

マイカレンダー

とんぴ

To Do

誕生日

他のカレンダー + ^

日本の祝日

利用規約 - プライバシー

適当な外出予定
5月 26日 (火曜日) ・ 午後8:00～8:30

茨木駅

とんぴ

予定なし

午後8時 適当な外出予定

**予定開始(外出予定時刻)
: 8:00
最終センサー反応: 7:55**

計算機科学

今日

Ryuさん
はじめまして！計算機科学です。
友だち追加ありがとうございます
👋

このアカウントでは、最新情報を定期的に配信していきます📧
どうぞお楽しみに🎁🌟

午後 8:03

外出を確認しました。いってらっしゃい！
予定: 適当な外出予定
開始: 20:00

午後 8:25

メッセージを入力

📎 📌 🗑️ 😊

カレンダーに場所が入力され、監視（外出30分前）が始まってから予定時間を過ぎた今現在も、玄関のセンサーが全く反応していない場合、**外出していない**とみなしてLINEで通知。

26 27 28

2026年 5月

日 月 火 水 木 金 土

26 27 28 29 30 1 2

3 4 5 6 7 8 9

10 11 12 13 14 15 16

17 18

24 25

31 1

作成

編集 削除 共有 閉じる

■ がい

5月 26日 (火曜日) ・ 午後9:30~10:30

リンクで招待

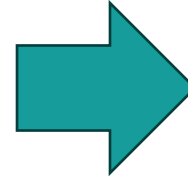
茨木市

日本、大阪府茨木市

30 分前

とんび

予定開始: 9:30
最終センサー反応: 9:59
(予定開始後remoの前を通るとLineに通知が来る)



計算機科学

開始: 20:45

午後 8:52

外出を確認しました。いってらっしゃい！
予定: がいしゅつ
開始: 21:12

午後 8:59

ここから未読メッセージ

【外出催促 第4段階 繰り返し警告】
予定: がい
開始: 21:30
現在: 21:59
玄関前の外出動作がまだ確認できていません。

午後 9:59

メッセージを入力

感想